
What do LearnBridge and LinCa change, and where do we stand?

Week 38 · two new learned competitors (ICML 2026, ECCV 2026) profiled against our schedules, plus our matched runs at $N=5-25$.

Diffusion feature caching

Weekly competitor watch · three sources · our own runs are labelled separately

Where we are, and what changed this week

ESTABLISHED (UNCHANGED)

- P1 Block outputs are temporally coherent, so a cached feature is a usable stand-in.
- Family A Fixed reuse (DeepCache, FORA) and calibrated reuse (TeaCache, ToCA).
- Family B History extrapolation with a fixed rule (TaylorSeer) at longer cache intervals.

WHERE WE ARE NOW

Two 2026 competitors move the correction itself into learned parameters, and both report quality that fixed rules do not reach at 5-7x. We now have matched runs against one of them.

Last week: schedule tuning only, no learned competitor in our comparison.

Four families, one schedule: where the competitors sit

A

Fixed reuse

DeepCache, FORA reuse a cached block output directly; TeaCache and ToCA add calibrated or dynamic caching decisions.

B

History forecast

TaylorSeer extrapolates to the next timestep from several cached features with a fixed Taylor-style rule.

C

Learned correction

LearniBridge fits a low-rank weight correction on the final block so a cached input reproduces the skipped full computation.

D

Learned decomposition

LinCa learns an invertible feature decomposition and matches a different prediction order to each component, per model and timestep segment.

OPEN GAP

Families C and D learn the correction; both still assume the caching schedule is given. That is where our schedule work sits.

Competitor 1: LearniBridge (ICML 2026)

A. INTUITION

Ask what correction makes the cached computation reproduce the skipped full computation, instead of forecasting the future feature.

B. MECHANISM

Fit a low-rank LoRA update on the linear layers of the final Transformer block; at skipped steps only that block runs. Calibration uses 3-5 prompts.

C. WHERE IT FAILS

- Needs a pre-calibration pass with full computation to record targets.
- Only the final block is calibrated, so it absorbs the shift of every bypassed block.

RELATION TO US

Its adapters presuppose a cache schedule; it changes which block runs, not when skips happen.

Competitor 2: LinCa (ECCV 2026)

A. INTUITION

Inside one feature, dimensions do not share the same temporal behaviour, so one prediction order for the whole feature is a mismatch.

B. MECHANISM

A lightweight invertible network splits the feature into components with distinct continuity, applies a matched prediction order to each, and maps them back with the exact inverse (under 0.2% extra parameters).

C. WHERE IT FAILS

- Needs trained decomposition and predictors per model and timestep segment.
- Reported near-lossless 5-7x is on FLUX, Qwen-Image and HunyuanVideo; not reproduced by us.

RELATION TO US

Our skip pattern still decides how far its prediction must reach.

The gap both competitors leave

GAP

Both competitors learn a better correction, but the caching schedule stays a fixed, hand-set interval; neither reports what happens when the schedule is data-dependent.

EVIDENCE

- LearniBridge evaluates a fixed full-compute interval N and calibrates one bridge for it.
- LinCa fixes the decomposition once per model and trains per segment, with the interval set externally.
- In our matched runs, the ranking between methods changes with the interval rather than staying fixed.

IMPLICATION

A schedule-aware competitor comparison is the right test, and it is the piece we can own.

Our approach: retarget the cached state, not the network

FORECAST (TAYLORSEER)

$Z_s \rightarrow Z_d$: predict the future feature first, then hope the frozen tail accepts it.

LEARNED WEIGHTS

(LEARNIBRIDGE)

$F_W \rightarrow F_W + \Delta W$: adapt the weights of the final block to the stale input.

THE IDEA

Ours keeps F_W frozen and edits the hidden state instead. Find $Z_{\sim s \rightarrow d}$ such that the frozen tail, given the true target condition, produces the target output.

The problem is not an incorrect timestep label; it is a stale hidden state. At the skipped step the tail still receives the correct target-step condition, so only the state is wrong.

$$Z_s \neq Z_d, \quad F_{\text{tail}}(Z_s, c_d) \neq F_{\text{tail}}(Z_d, c_d)$$

$$F_{\text{tail}}^{\text{frozen}}(\tilde{Z}_{s \rightarrow d}, c_d) \approx F_{\text{tail}}^{\text{frozen}}(Z_d, c_d)$$

The retargeter: three inputs, one small MLP

THREE INPUTS PER TOKEN

- Local. the cached token state z_i at step s .
- Global. an image-wide summary c_s , the mean of the normalized tokens.
- Transition. an embedding e_t of the step pair (t_s, t_d) and the gap Δ .

FUSE, THEN PREDICT THE CORRECTION

Normalize the token, fuse the three signals through a SiLU layer, and predict the correction with a two-layer MLP. The retargeted token is the cached token plus that correction.

$$x_i = \text{RMSNorm}(z_i), \quad c_s = \frac{1}{L} \sum_{i=1}^L x_i$$

$$e_t = E(t_s, t_d, \Delta), \quad h_i = \text{SiLU}(W_z x_i + W_c c_s + e_t)$$

$$\Delta z_i = W_2 \text{SiLU}(W_1 h_i), \quad \tilde{z}_{s,i} = z_{s,i} + \Delta z_i$$

What is frozen, and what the loss actually asks for

GAP

Only the retargeter is trained. Everything below the edited state stays frozen, and the tail still runs with the true target-step condition.

EVIDENCE

- The frozen tail consumes the retargeted state:
 $(H_d, v_d) = F_{\text{tail}}(Z_{\sim_d}, c_d)$.
- The training loss matches the teacher velocity;
no term pulls the retargeted state toward Z_d .
- The edited state only has to be a useful
pre-image for the tail, not a copy of the future
state.

IMPLICATION

Metrics that measure distance to Z_d can
punish the method while the tail output
improves; judge the retargeter by the tail
output, not by state distance.

$$\mathcal{L}_{\text{vel}} = \|\hat{v}_d - v_d^*\|_2^2, \quad \text{no } \|\tilde{Z}_{s \rightarrow d} - Z_d\|^2 \text{ term}$$

What our matched runs actually show

OBSERVATION

At matched cache intervals, Global-Local leads the fidelity metrics (PSNR, SSIM, HPSv2.1) at $N=5$, 7 and 10, while LearnBridge-100 leads CLIP at $N=7$ and $N=10$; at $N=25$ all three collapse together.

INTERPRETATION

The learned bridge competes on semantic alignment rather than pixel fidelity in our setting, and no method survives the 25-step interval. Our retargeter is trained only through the tail output, so its gain shows up on quality metrics rather than on distance-to-original fidelity.

CLAIM

C1 **weakened** A learned low-rank calibration dominates our baseline at matched intervals. Our runs do not support this as stated.

Our matched runs: N=5 and N=7

Method	CLIP $\uparrow$	IR $\uparrow$	HPSv2.1 $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$	s/ảnh $\downarrow$	Speed $\uparrow$
Original FLUX	32.65	1.00	30.40	∞	1	22.88	1.00x
Cache thường (N=5)	32.63	0.94	29.44	14.70	0.55	4.95	4.63x
Global-Local (N=5) _(ours)	32.79	1.01	30.00	15.21	0.57	4.98	4.60x
LearniBridge-100 (N=5)	32.66	0.98	29.53	15.07	0.55	5.09	4.49x
Cache thường (N=7)	32.28	0.83	28.24	14.14	0.52	4.04	5.66x
Global-Local (N=7) _(ours)	32.63	0.92	29.10	14.73	0.55	4.09	5.60x
LearniBridge-100 (N=7)	32.76	0.92	28.44	14.49	0.54	4.20	5.45x

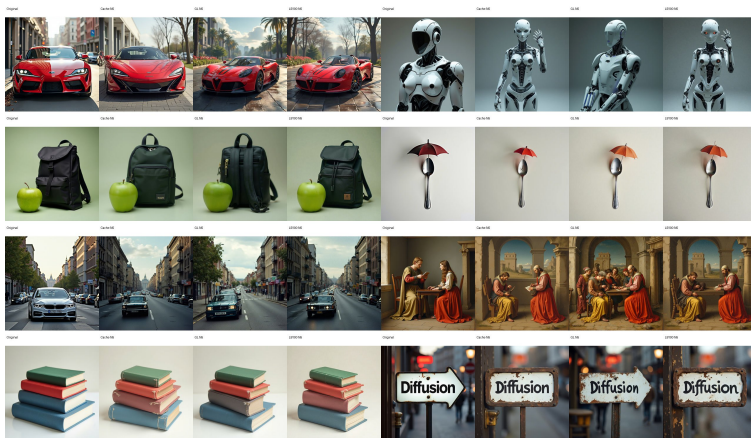
Ours = the retargeter (Global-Local). Green marks the best value in each interval, computed separately for N=5 and N=7; arrows say whether higher or lower is better. PSNR/SSIM are measured against Original FLUX, which is a reference row and is never highlighted.

Our matched runs: N=10 and N=25

Method	CLIP ↑	IR ↑	HPSv2.1 ↑	PSNR ↑	SSIM ↑	s/ảnh ↓	Speed ↑
Cache thường (N=10)	31.72	0.66	26.36	13.77	0.48	2.70	8.48x
Global-Local (N=10) _(ours)	31.77	0.76	27.76	14.46	0.52	2.75	8.32x
LearniBridge-100 (N=10)	32.16	0.73	26.68	14.10	0.47	2.88	7.96x
Cache thường (N=25)	23.08	-1.48	13.71	13.12	0.42	1.36	16.87x
Global-Local (N=25) _(ours)	22.99	-1.28	15.68	14.26	0.49	1.41	16.24x
LearniBridge-100 (N=25)	23.09	-1.51	13.72	13.47	0.45	1.55	14.77x

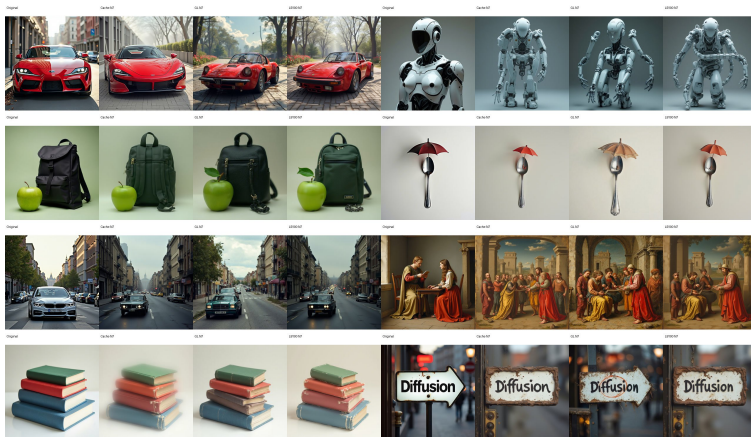
Green marks the best value within each interval (N=10, then N=25). At N=25 every method is far from Original FLUX (IR below zero, HPSv2.1 near half), so N=25 is a failure regime for all three and should not be ranked.

Qualitative, N=5: Original / Cache / ours / LearniBridge-100



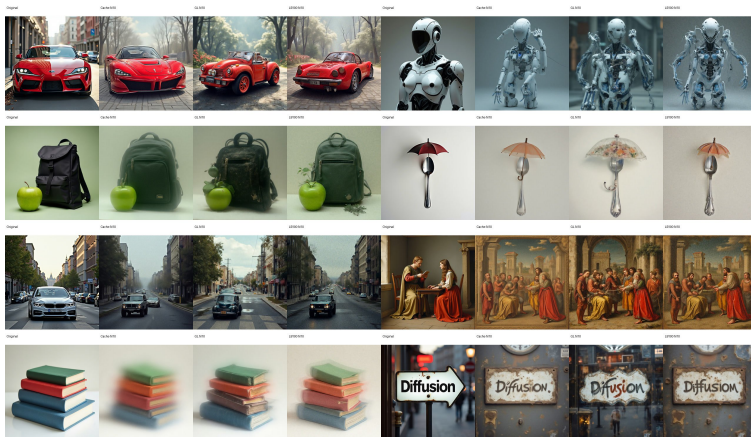
Sheet split in two blocks: rows 1-4 on the left, rows 5-8 on the right. Columns are Original, Cache (N=5), Global-Local (ours), LearniBridge-100.

Qualitative, N=7: Original / Cache / ours / LearniBridge-100



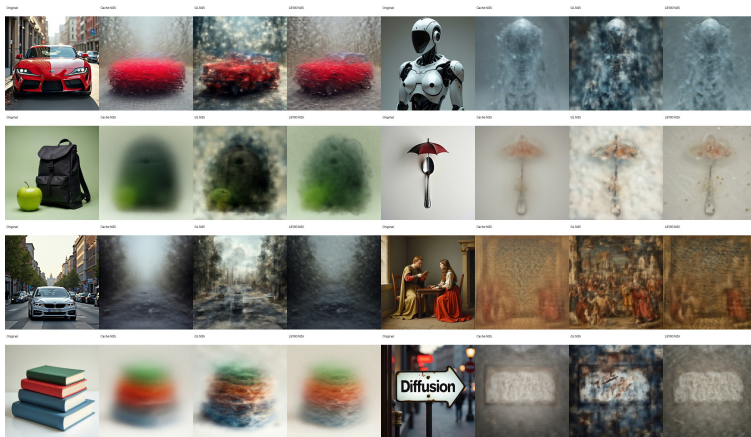
Rows 1-4 on the left, rows 5-8 on the right. At N=7 the plain cache starts to drift on structure while both learned methods keep the layout.

Qualitative, N=10: Original / Cache / ours / LearniBridge-100



Rows 1-4 on the left, rows 5-8 on the right. Failures are no longer uniform: which method drifts depends on the prompt.

Qualitative, N=25: Original / Cache / ours / LearniBridge-100



Rows 1-4 on the left, rows 5-8 on the right. At N=25 all four columns are visibly different from Original; this sheet documents the failure regime, not a comparison.

What we still do not know

CURRENT LIMITATIONS

- LinCa is profiled from its paper only; we have not reproduced it in our matched setting.
- Our runs cover one model (FLUX) and one prompt set, so cross-model ranking is unmeasured.
- PSNR/SSIM against Original FLUX penalise every accelerator; they are fidelity, not quality.
- $N=25$ is a failure regime for all three methods, so the interval range where learned corrections pay off is still narrow.

OPEN QUESTIONS

- Does LinCa's learned decomposition close the $N=10-25$ gap that no current method survives?
- Is the CLIP advantage of LearniBridge-100 at $N=7/10$ reproducible across prompt sets?
- Would a data-dependent schedule change which of these competitors wins?